

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Вычислительная математика»
Тема: Решение нелинейных уравнений. Метод Ньютона.

Студент гр. 4342

Алюкин Г.А.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2025

Цель работы.

В случае, когда известно хорошее начальное приближение решения уравнения $f(x) = 0$, эффективным методом повышения точности является метод Ньютона. Он состоит в построении итерационной последовательности $x_{n+1} = x_n - f(x_n)/f'(x_n)$, сходящейся к корню уравнения $f(x)=0$. Достаточные условия сходимости метода формулируются теоремой, приведенной в [1,7].

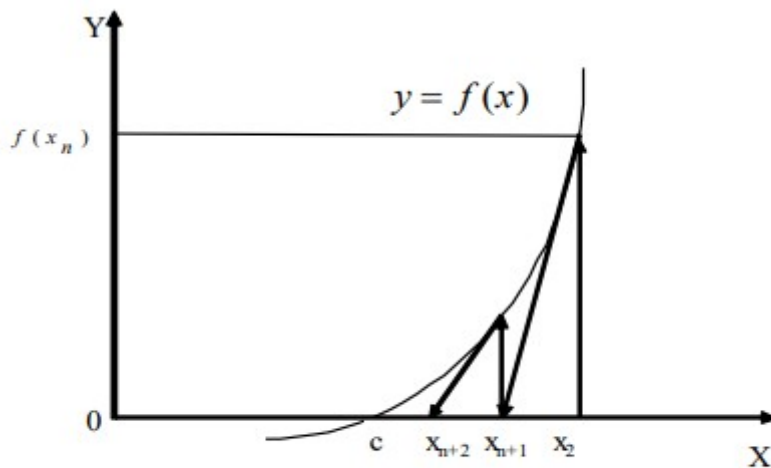


Рис.2

Метод Ньютона допускает простую геометрическую интерпретацию (рис. 2). Если через точку с координатами $(x_n, f(x_n))$ провести касательную, то абсцисса точки пересечения этой касательной с осью Ox будет очередным приближением x_{n+1} корня уравнения.

Для оценки погрешности n -го приближения корня предлагается пользоваться неравенством

$$|c - x_n| \leq \frac{M_2}{2m_1} |x_n - x_{n-1}|^2,$$

где M_2 -наибольшее значение модуля второй производной на отрезке $[a,b]$; m_1 -наименьшее значение модуля первой производной на отрезке $[a,b]$. Таким

образом, если $|x_n - x_{n-1}| < \varepsilon$, то $|c - x_n| \leq \frac{M_2 \varepsilon^2}{2m_1}$. Это означает, что при хорошем начальном приближении корня после каждой итерации число верных десятичных знаков в очередном приближении удваивается, т.е. процесс сходится очень быстро (имеет место квадратическая сходимость). Из указанного следует, что при

необходимости нахождения корня с точностью ε итерационный процесс можно прекращать, когда

$$|x_n - x_{n-1}| < \varepsilon_0 = \sqrt{2m_1\varepsilon / M_2}. \quad (3.1)$$

Рассмотрим один шаг итераций. Если на $(n-1)$ -м шаге очередное приближение x_{n-1} не удовлетворяет условию окончания процесса, то вычисляются величины $f(x_{n-1}), f'(x_{n-1})$ и следующее приближение корня $x_n = x_{n-1} - f(x_{n-1}) / f'(x_{n-1})$. При выполнении условия (3.1) величина x_n принимается за приближенное значение корня s , вычисленное с точностью ε .

Порядок выполнения лабораторной работы №5:

1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки [Left, Right], на которых функция $f(x)$ удовлетворяет условиям применимости метода Ньютона).

2) Составить подпрограмму - функции вычисления функции $f(x)$, $f'(x)$ предусмотрев округление их значений с заданной точностью Delta.

3) Составить головную программу, вычисляющую корень уравнения $f(x)=0$ и содержащую обращение к подпрограммам, $f(x)$, $f'(x)$, Round, NEWTON и индикацию результатов

4) Выбрать начальное приближение корня x_0 из [Left, Right] так, чтобы $f(x) f''(x) > 0$.

5) Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных.

Для приближенного вычисления корней уравнения $f(x) = 0$ методом Ньютона предназначена программа - функция NEWTON, текст которой представлен в подразделе 3.7.

Выполнение работы.

1. Отделение корня уравнения (т.е. нахождение отрезков [Left, Right], на которых функция удовлетворяет условиям применимости метода).

Функция:

$$f(x) = x^2 - 5\sin(x)$$

Для отделения корней воспользуемся свойством непрерывной функции: если на концах отрезка функция имеет разные знаки, то внутри отрезка существует хотя бы один корень.

Подставляем разные значения x и смотрим на знак результата:

x	Результат $f(x)$	Знак
-1	≈ 5.2	+
0	0	Корень
1	≈ -3.2	-
2	≈ -0.55	-
3	≈ 8.3	+

Вычисляем значения функции:

$$f(2) = 2^2 - 5\sin(2) \approx -0.55 \quad - \text{отрицательное}$$

$$f(3) = 3^2 - 5\sin(3) \approx 8.30 \quad - \text{положительное}$$

- Непрерывность: Функция непрерывна как разность двух непрерывных функций:

- Производные:

$f'(x) = 2x - 5\cos(x)$. На $[2, 3]$ производная положительна ($2x \geq 4$), значит функция монотонно возрастает.

$f''(x) = 2 + 5\sin(x)$. На $[2, 3]$ синус положителен (2 четверть тригонометрического круга), следовательно $f''(x) > 0$ на всём отрезке $[2, 3]$.

Функция не меняет направление выпуклости.

Между $x = 2$ (функция ниже нуля) и $x = 3$ (выше нуля) происходит изменение знака. Корень лежит на отрезке $[2, 3]$, Функция должна быть

непрерывна и менять знак. Это выполняется, следовательно берём данный отрезок в качестве начального для метода Ньютона.

$[Left; Right] = [2; 3]$.

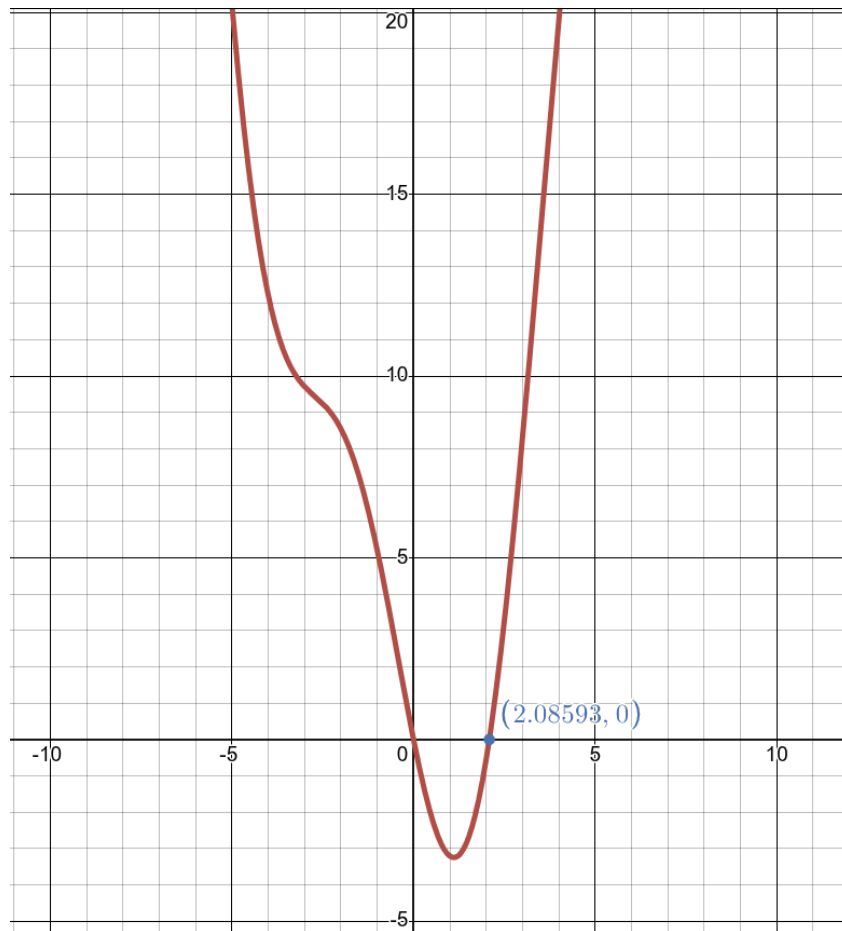


Рисунок 1 — График функции

2. Подпрограмма вычисления функции $f(x)$, $f'(x)$.

Для работы метода Ньютона необходима подпрограмма, вычисляющая $f(x)$, $f'(x)$ предусматривая округление их значений с заданной точностью Δ для произвольного x .

Создана подпрограмма находящаяся в файле `lab5.cpp` в Приложении А.

```
double F(double x){  
    extern double delta;  
    double s;  
    s = x * x - 5.0 * sin(x); // Значение функции  
    s = Round(s, delta); // Округление с точностью delta
```

```

    return s;
}

```

Функция `F(double x)` реализует вычисление значения исследуемого уравнения с имитацией погрешности вычислений. Результат вычисления передается в функцию `Round`, которая округляет его до шага, определяемого параметром `delta`.

3. Составить главную программу, вычисляющую корень уравнения $f(x)=0$ и содержащую обращение к подпрограммам `f(x)`, `f'(x)`, `Round`, `NEWTON` и индикацию результатов.

Создана головная функция `int main()` находящаяся в файле `lab5.cpp` в Приложении А.

4. Выбрать начальное приближение корня x_0 из `[Left, Right]` так, чтобы $f(x) \times f''(x) > 0$.

$$f'(x) = 2x - 5\cos(x)$$

$$f''(x) = 2 + 5\sin(x)$$

На интервале `[2, 3]` (вторая четверть тригонометрического круга) значение $\sin(x)$ всегда положительно. Следовательно $f'(x) > 0$ на всем отрезке `[2, 3]`. График функции всегда «выпуклый вниз»

Условие Фурье гласит: начальное приближение x_0 должно выбираться из того конца отрезка, где знак функции совпадает со знаком второй производной.

$$f(x_0) * f''(x_0) > 0$$

Проверим обе границы нашего отрезка:

1. $x = 2$

$$f(2) \approx -0,54$$

- $f(2) < 0$ (отрицательно)
- $f'(2) > 0$ (положительно)
- Произведение $f(2) * f'(2) < 0$. Условие не выполнено.

2. $x = 3$:

$$f(3) \approx 8,29$$

- $f(3) > 0$ (положительно)
- $f'(3) > 0$ (положительно)
- Произведение $f(3) * f'(3) > 0$. Условие выполнено.

Вывод: В качестве начального приближения необходимо выбрать $x_0 = 3$.

4. Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

На основе выполнения алгоритма метода Ньютона получаем следующие результаты:

Точность (ϵ)	Корень (x)	Число итераций (N)
10^{-1} (0.1)	2.0859799	3
10^{-2} (0.01)	2.0859799	3
10^{-3}	2.0859346	4
10^{-4}	2.0859346	4
10^{-5}	2.0859346	4
10^{-6}	2.0859346	4

Построение графика зависимости N от ϵ .

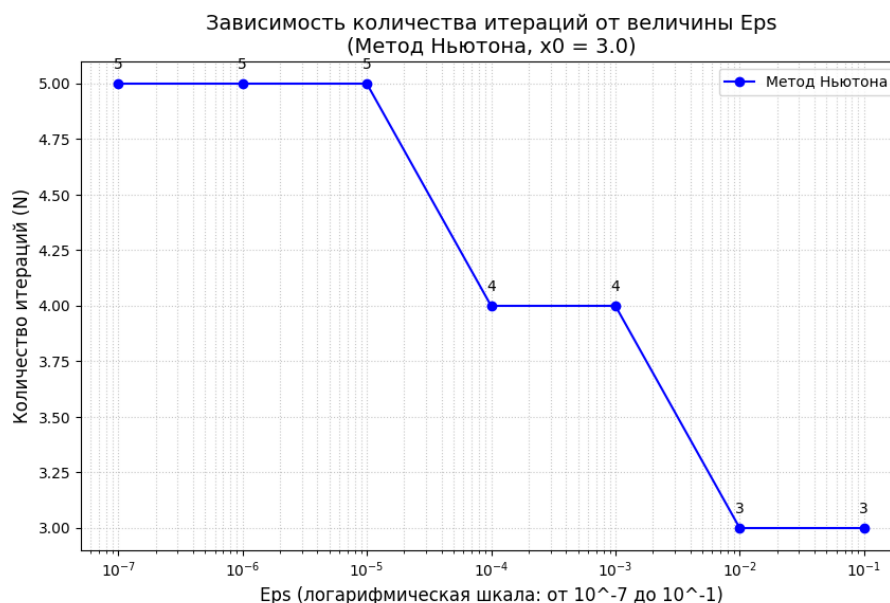


Рисунок 2 — График зависимости N от точности вычислений

5. Исследование чувствительности метода к ошибкам в исходных данных (моделирование Round)

Чтобы исследовать чувствительность метода к ошибкам в исходных данных, было проведено исследование с разными Delta и Eps. Результаты вычислений приведены ниже.

Delta	Eps	Значение x	N
10^{-1}	10^{-1}	2.0788699	3
	10^{-2}	2.0788699	3
	10^{-3}	2.0788699	3
	10^{-4}	2.0788699	3
	10^{-5}	2.0788699	3
	10^{-6}	2.0788699	3
10^{-3}	10^{-1}	2.0859349	3
	10^{-2}	2.0859349	3
	10^{-3}	2.0859349	3
	10^{-4}	2.0859349	3
	10^{-5}	2.0859349	3
	10^{-6}	2.0859349	3
10^{-5}	10^{-1}	2.0859793	3
	10^{-2}	2.0859793	3
	10^{-3}	2.0859341	4
	10^{-4}	2.0859341	4
	10^{-5}	2.0859341	4
	10^{-6}	2.0859341	4
10^{-7}	10^{-1}	2.0859799	3
	10^{-2}	2.0859799	3
	10^{-3}	2.0859346	4
	10^{-4}	2.0859346	4
	10^{-5}	2.0859346	4
	10^{-6}	2.0859346	4

Исследование показало, что достижимая точность поиска корня напрямую ограничена погрешностью вычисления функции (Delta).

Таким образом применение функции Round позволяет исследовать зависимость точности решения от ошибок округления. Сравнительный анализ результатов при изменении шага Delta дает возможность определить чувствительность алгоритма к точности представления чисел и выявить границы метода.

Вывод.

В ходе выполнения лабораторной работы был изучен и программно реализован метод Ньютона для нахождения корня нелинейного уравнения $f(x)=x^2 - 5.0 * \sin(x)$ на отрезке $[2, 3]$. Исследован график зависимости числа итераций N от точности Eps . Таким образом, метод Ньютона является крайне надежным и устойчивым алгоритмом, однако требует строгого соблюдения условий выбора начального приближения для предотвращения расходимости процесса.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lb5.cpp:

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include "methods.h"
#define __NEWTON

double delta;

double F(double x){
    double res = x * x - 5.0 * sin(x);
    return Round(res, delta);
}

double derivativeF(double x) {
    double res = 2.0 * x - 5.0 * cos(x);
    return Round(res, delta);
}

int main(){
    double x0, eps, x;
    int k;
    float x0_in, eps1, delta1;

    printf("Лабораторная работа №5. Вариант 1\n");
    printf("Функция: f(x) = x^2 - 5*sin(x)\n");

    printf("Введите начальное приближение x0: "); // 3.0
    scanf("%f", &x0_in);
    x0 = 3.0;
```

```

        printf("Введите точность вычисления корня (Eps): "); //
0.0001
        scanf("%f", &eps1);
        eps = eps1;

        printf("Введите точность задания функции (Delta): "); //
0.00000001
        scanf("%f", &delta1);
        delta = delta1;

        x = NEWTON(x0, eps, k);

        printf("Результаты:\n");
        printf("x = %7.7f\n", x);
        printf("Число итераций N = %d\n", k);
        return 0;
}

```

Файл methods.cpp:

```

#include <stdio.h>
#include <math.h>
#include <stdlib.h>

extern double F(double);
extern double derivativeF(double);

double NEWTON(double X, double Eps, int &N)
{
    double Y, Y1, DX;
    N = 0;
    do
    {
        Y = F(X);
        if (Y == 0.0)
            return (X);
    }
}

```

```

        Y1 = derivativeF(X);
        if (Y1 == 0.0)
        {
            puts("Производная обратилась в ноль\n");
            exit(1);
        }
        DX = Y / Y1;
        X = X - DX;
        N++;
    } while (fabs(DX) > Eps);
    return (X);

}

/*
ФУНКЦИЯ ОКРУГЛЕНИЯ
X - число
Delta - шаг округления (например, 0.01)
*/
double Round(double X, double Delta)
{
    if (Delta <= 1E-9) {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0)
        return (Delta * (long((X / Delta) + 0.5)));
    else
        return (Delta * (long((X / Delta) - 0.5)));
}

```

Файл methods.h

```

#include "methods.cpp"

extern double F(double);

```

```
double Round (double X,double Delta);  
double HORDA(double Left, double Right, double Eps, int &N);  
double NEWTON(double X, double Eps, int &N);
```